COMPLETE EXAM NOTES

Unit 5

The Transport Layer, DNS, SMTP & HTTP

Transport Layer Protocols · Domain Name System

Email Protocols · World Wide Web Architecture

Table of Contents

Chapter 6: The Transport Layer

6.1 The Transport Service

6.1.1 Services Provided to the Upper Layers

> **Primary Objective**
>
> The transport layer provides **efficient, reliable, and cost-effective data transmission service** to application-layer processes. It uses the services of the network layer to achieve this goal.

The Transport Entity is the software and/or hardware within the transport layer that performs data transmission. It can be located in:

Table 6.1: Transport Entity Locations

Location	Description
Operating system kernel	Most common on the Internet
Library package bound into network applications	Most common on the Internet
Separate user process	Less common
Network interface card	Less common

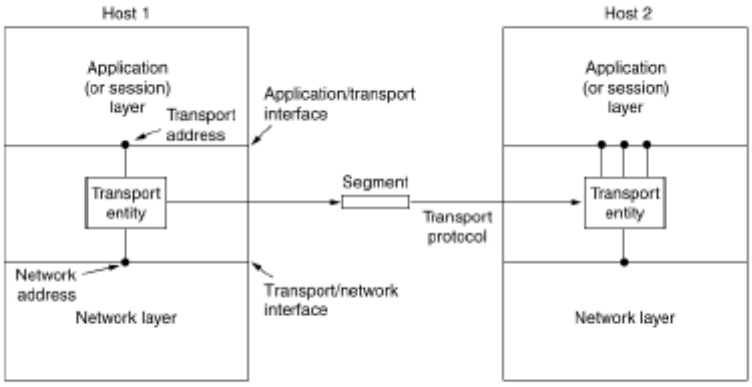


Figure 6-1. The network, transport, and application layers.

Figure 6.1: *The network, transport, and application layers. The transport entity resides in the transport layer and interfaces with the application layer above (via transport address) and the network layer below (via network address). Communication between transport entities occurs via the transport protocol, exchanging segments.*

> **Exam Tip**
>
> The transport entity is the key boundary: it uses **network addresses** below and **transport addresses (ports)** above. Segments are the unit of communication between transport entities.

Types of Transport Service

Table 6.2: Transport Service Types

Type	Characteristics
Connection-oriented	Three phases: establishment, data transfer, and release. Addressing and flow control similar to connection-oriented network service.
Connectionless	Very similar to connectionless network service.

> **Important**
>
> It is difficult to provide a connectionless transport service on top of a connection-oriented network service, because setting up a connection to send a single packet and tearing it down is inefficient.

Why a Distinct Transport Layer?

Central Question: If transport and network layer services are so similar, why have two layers?

Table 6.3: Transport vs Network Layer

Aspect	Transport Layer	Network Layer
Execution location	Runs entirely on users' machines	Mostly runs on routers (carrier-operated)
User control	Users have full control	Users have no real control
Service improvement	Can compensate for poor network service	Cannot be modified by end users

> **Key Insight**
>
> The transport layer makes it possible for the transport service to be **more reliable than the underlying network**. If packets are lost, the transport entity can detect and retransmit. If a network connection is terminated mid-transmission, it can set up a new connection and pick up from where it left off.

Network Independence: Transport primitives hide network service differences. Changing the network merely requires replacing one set of library procedures with another. Application programmers write code to standard primitives, and it works across Ethernet, WiMAX, etc.

> **Provider-User Distinction**
>
> - **Layers 1-4:** The transport service **provider**
> - **Layers 5+:** The transport service **user**
>
> The transport layer forms the major boundary. **It is the level that applications see.**

6.4 The Internet Transport Protocols: UDP

Table 6.4: Internet Transport Protocols

Protocol	Type	Characteristics

UDP (User Datagram Protocol)	Connectionless	Does almost nothing beyond sending packets between applications
TCP (Transmission Control Protocol)	Connection-oriented	Connections, reliability, retransmissions, flow control, congestion control

6.4.1 Introduction to UDP

> **UDP Definition**
>
> UDP is a **connectionless transport protocol** (RFC 768) that lets applications send encapsulated IP datagrams **without establishing a connection**.

UDP Segment Structure

Each UDP segment has an **8-byte header** followed by the payload data.

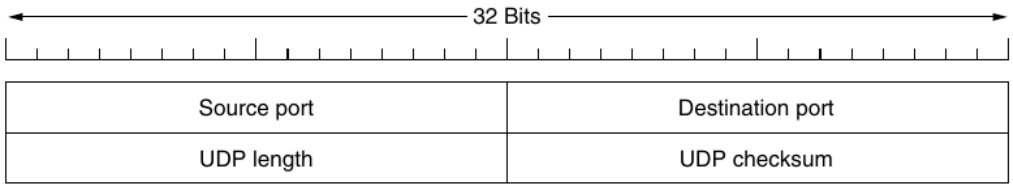


Figure 6-27. The UDP header.

Figure 6.27: *The UDP header format (8 bytes). Fields: Source port (16 bits), Destination port (16 bits), UDP length (16 bits), UDP checksum (16 bits).*

Table 6.5: UDP Header Fields

Field	Size	Description
Source port	16 bits	Identifies sending endpoint; needed for replies
Destination port	16 bits	Identifies receiving endpoint
UDP length	16 bits	Includes 8-byte header + data. Min=8, Max=65,515 bytes
UDP checksum	16 bits	Optional checksum for extra reliability

The Port Mechanism

When a UDP packet arrives, its payload is handed to the process attached to the destination port via the **BIND primitive**. Think of ports as **mailboxes** that applications rent to receive packets.

Main value of UDP over raw IP: The addition of source and destination ports enables **demultiplexing** -- delivering packets to the correct application.

UDP Checksum

- **Optional** but provided for extra reliability
- Covers: UDP header + UDP data + **conceptual IP pseudoheader**
- Stored as 0 means "checksum not computed"; a true computed 0 is stored as all 1s

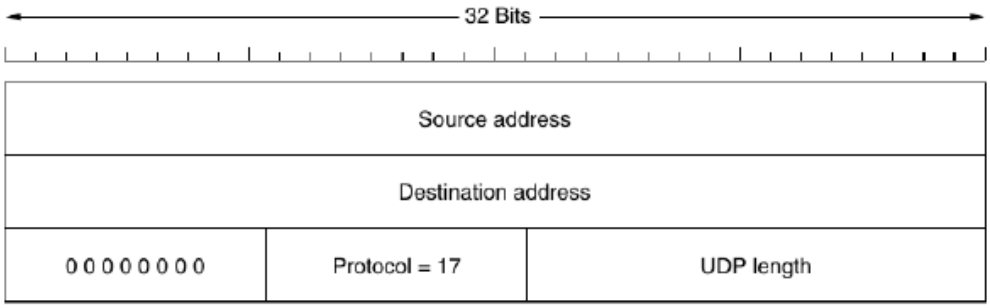


Figure 6-28. The IPv4 pseudoheader included in the UDP checksum.

Figure 6.28: *The IPv4 pseudoheader included in the UDP checksum. Contains: Source address (32 bits), Destination address (32 bits), Padding (8 bits = 0), Protocol (8 bits = 17 for UDP), UDP length (16 bits).*

> **Protocol Hierarchy Violation**
>
> Including the pseudoheader violates protocol hierarchy since IP addresses belong to the IP layer, not UDP. But it helps **detect misdelivered packets**. TCP uses the **same pseudoheader**.

What UDP Does NOT Do

Table 6.6: Features UDP Lacks

Missing Feature	Consequence
Flow control	Up to user processes

Congestion control	Up to user processes
Retransmission of bad segments	Up to user processes

What UDP Does

Table 6.7: What UDP Provides

Feature	Description
Interface to IP	Provides access to IP protocol
Demultiplexing	Uses ports to direct packets to correct process
Optional error detection	Via checksum mechanism

> **UDP Summary**
>
> UDP provides an interface to IP with demultiplexing via ports and optional end-to-end error detection. That is all it does -- and for many applications (DNS, streaming), that is exactly what is needed.

6.5 The TCP Protocol

6.5.3 The TCP Protocol Overview

> **Key Feature**
>
> Every byte on a TCP connection has its own **32-bit sequence number**. This dominates the entire protocol design.

Segment Structure: TCP entities exchange data in **segments**:

- Fixed **20-byte header** (plus optional part)
- Followed by **zero or more data bytes**

Table 6.8: Limits on Segment Size

Limit	Constraint
IP payload limit	Each segment (including TCP header) must fit in 65,515-byte IP payload
MTU	Must fit in MTU at sender and receiver for single unfragmented packet

Practical MTU: Generally **1,500 bytes** (Ethernet payload size), defining the upper bound on segment size.

Path MTU Discovery

Modern TCP uses **Path MTU discovery** (RFC 1191) with ICMP error messages to find the smallest MTU on the path, then adjusts segment size downward to avoid fragmentation.

Basic TCP Protocol: Sliding Window

1. Sender transmits a segment and **starts a timer**
2. Receiver sends back a segment with: **acknowledgement number** (next expected seq) + **remaining window size**
3. If timer expires before ACK, sender **retransmits**

> **Complexity Warning**
>
> Segments can arrive **out of order**, be **delayed** causing unnecessary retransmissions, or have **retransmission ranges** that differ from originals. TCP must track each byte with its unique offset to handle all cases.

6.5.4 The TCP Segment Header

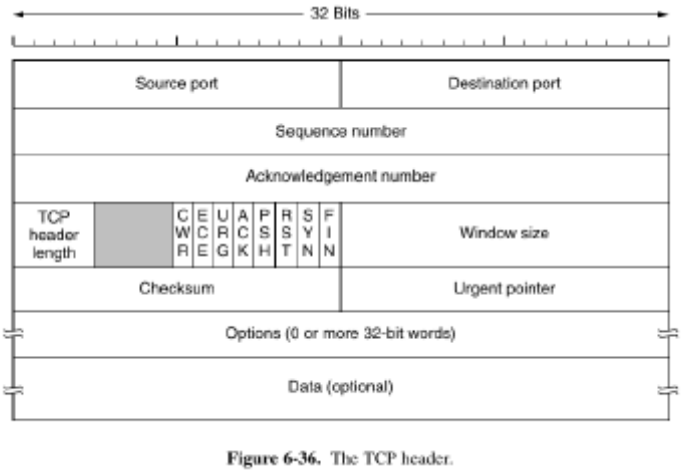


Figure 6-36. The TCP header.

Figure 6.36: *The TCP header. Fixed 20-byte header + optional options. Max data = 65,535 - 20 (IP) - 20 (TCP) = 65,495 bytes.*

Field-by-Field Dissection

Table 6.9: TCP Header Fields

Field	Size	Description
Source port	16 bits	Local endpoint on sending host
Destination port	16 bits	Local endpoint on receiving host
Sequence number	32 bits	Sequence number of first data byte
Acknowledgement number	32 bits	Next expected in-order byte (cumulative ack)
TCP header length	4 bits	Number of 32-bit words in TCP header
Reserved/Unused	4 bits	Unused for 30+ years (testament to good design)
Flags (8 bits)	8 x 1 bit	CWR, ECE, URG, ACK, PSH, RST, SYN, FIN
Window size	16 bits	Bytes allowed to be sent (flow control)

Checksum	16 bits	Covers header, data, pseudoheader (mandatory)
Urgent pointer	16 bits	Byte offset of urgent data
Options	Variable	TLV-encoded; up to 40 bytes
Data	Variable	Optional payload (up to 65,495 bytes)

> **Memory Trick**
>
> Remember the 8 flags with: **U**nited **AP**eaceful **S**ociety **R**equires **S**ome **F**riends **C**alm -- URG, ACK, PSH, RST, SYN, FIN, ECE, CWR

The 5-Tuple Connection Identifier

Table 6.10: 5-Tuple Components

Component	Value
Protocol	TCP
Source IP address	Host A's IP
Source port	Port on Host A
Destination IP address	Host B's IP
Destination port	Port on Host B

Flag Details

Table 6.11: TCP Flag Functions

Flag	Name	Function
CWR	Congestion Window Reduced	Signal to receiver that sender has slowed down (ECN)
ECE	ECN-Echo	Signal to sender to slow down when receiver gets congestion indication
URG	Urgent	Urgent pointer is valid (seldom used)
ACK	Acknowledgement	Acknowledgement number is valid (nearly all packets)

PSH	Push	Deliver data to application immediately, don't buffer
RST	Reset	Abruptly reset or reject a confused connection
SYN	Synchronize	Establish connection. SYN=1,ACK=0 = request; SYN=1,ACK=1 = reply
FIN	Finish	Release connection (sender has no more data)

Window Size and Flow Control

TCP uses a **variable-sized sliding window** for flow control. The Window size field tells how many bytes may be sent starting at the acknowledged byte.

> **Window Size = 0**
>
> A window size of 0 is **legal**. It says: bytes up to (Ack number - 1) have been received, but the receiver is not ready for more data yet. The receiver can later grant permission by sending a segment with the same ACK number and a nonzero window size.

Key difference from Chap 3 protocols: Acknowledgements and permission to send are **completely decoupled** in TCP, giving additional flexibility.

Checksum

Mandatory in TCP (unlike UDP where it's optional). Computed the same way as UDP except: protocol number = **6** for TCP.

TCP Options

Table 6.12: Common TCP Options

Option	Purpose	Key Detail
MSS	Maximum Segment Size	Default 536-byte payload; hosts must accept 556 bytes
Window Scale	Extend window beyond 64 KB	Shift window up to 14 bits left, allowing up to 2^30 bytes
Timestamp	RTT estimation + PAWS	Echoed by receiver; prevents sequence number wraparound

SACK	Selective Acknowledgement	Reports received ranges beyond cumulative ack (RFC 2108)

> **Numerical Tip**
>
> On an **OC-12 line (600 Mbps)**, a full 64-KB window takes **< 1 msec** to output. With a 50 msec RTT, the sender is idle **> 98%** of the time without window scaling. Window scale enables up to **2^30 bytes (~1 GB)** windows.

Chapter 7: DNS, SMTP & The World Wide Web

7.1 The Domain Name System (DNS)

Motivation for DNS

> **The Problem**
>
> IP addresses are **hard for people to remember**. If a company moves its server to a different machine with a different IP, everyone needs to be told the new address.

> **The Solution**
>
> High-level, readable names **decouple machine names from machine addresses**. Example: `www.cs.washington.edu` works regardless of its current IP address. DNS provides the mechanism to convert names to network addresses.

Historical Evolution

ARPANET era: A single file `hosts.txt` listed all computer names and IP addresses. Every night, all hosts fetched it.

Table 7.1: Why hosts.txt Failed

Problem	Explanation
File size	Would be too large for millions of PCs
Name conflicts	Would occur constantly without central management
Central management	Infeasible for a huge international network

DNS was invented in 1983 and has been a key part of the Internet ever since.

Essence of DNS

1. A **hierarchical, domain-based naming scheme**

2. A **distributed database system** implementing this naming scheme

Primary use: Mapping host names to IP addresses. Defined in **RFCs 1034, 1035, 2181**.

The Resolver Mechanism

Table 7.2: DNS Resolution Steps

Step	Action
1	Application calls the **resolver** library, passing the name
2	Resolver sends a **query** to a **local DNS server**
3	Local DNS server looks up the name and returns the **IP address**
4	Resolver returns the IP address to the caller

Transport: **UDP packets**. With the IP address, the program establishes a TCP connection or sends UDP packets.

7.1.1 The DNS Name Space

DNS uses a **hierarchical naming scheme** similar to the postal system (country, state, city, street, name).

ICANN (Internet Corporation for Assigned Names and Numbers) manages the top of the hierarchy, created in 1998.

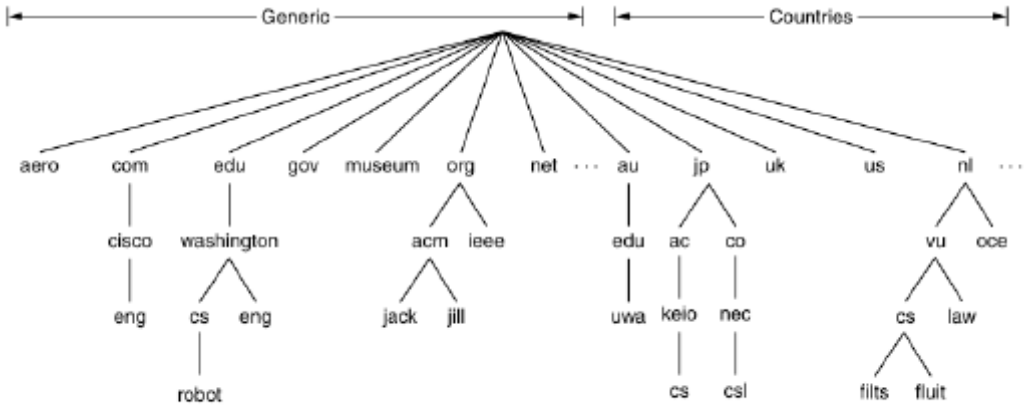


Figure 7-1. A portion of the Internet domain name space.

Figure 7.1: *A portion of the Internet domain name space. The tree has generic top-level domains (com, edu, gov, org, net, etc.) and country domains (au, jp, uk, us, nl, etc.). Leaves represent domains with hosts.*

Top-Level Domain Types

Table 7.3: TLD Categories

Type	Description
Generic domains	Original domains from 1980s + domains via ICANN applications
Country domains	One entry per country (ISO 3166). Internationalized domains (Arabic, Cyrillic, Chinese) introduced in 2010.

Domain	Intended use	Start date	Restricted?
com	Commercial	1985	No
edu	Educational institutions	1985	Yes
gov	Government	1985	Yes
int	International organizations	1988	Yes
mil	Military	1985	Yes
net	Network providers	1985	No
org	Non-profit organizations	1985	No
aero	Air transport	2001	Yes
biz	Businesses	2001	No
coop	Cooperatives	2001	Yes
info	Informational	2002	No
museum	Museums	2002	Yes
name	People	2002	No
pro	Professionals	2002	Yes
cat	Catalan	2005	Yes
jobs	Employment	2005	Yes
mobi	Mobile devices	2005	Yes
tel	Contact details	2005	Yes
travel	Travel industry	2005	Yes
xxx	Sex industry	2010	No

Figure 7-2. Generic top-level domains.

Figure 7.2: *Generic top-level domains. Some are restricted (edu, gov, mil, aero, etc.) and some are open (com, net, org, biz, info, etc.).*

Domain Name Syntax

- Each domain is named by the **path upward** from it to the (unnamed) root
- Components separated by **periods** ("dot")
- Example: `eng.cisco.com` (vs. UNIX-style `/com/cisco/eng`)
- **Case-insensitive**: edu, Edu, EDU are the same
- Component names: up to **63 characters**
- Full path names: must not exceed **255 characters**

Table 7.4: Absolute vs Relative Names

Type	Syntax	Characteristic

Relative	No trailing period (eng.cisco.com)	Must be interpreted in context

Domain Placement

US organizations typically under **generic domains**; organizations outside the US typically under **their country domain**. Large companies often register under multiple TLDs (e.g., sony.com, sony.net, sony.nl).

> **Exam Tip**
>
> Naming follows **organizational boundaries**, not physical networks. Two departments on the same LAN can have different domains. One department split across buildings normally shares the same domain.

7.1.2 Domain Resource Records

> **Fundamental Concept**
>
> Every domain has a **set of resource records** associated with it. These records **are** the DNS database. The primary function of DNS is to map domain names onto resource records.

Resource Record Format

A resource record is a **five-tuple**:

```
Domain_name  Time_to_live  Class  Type  Value
```

Table 7.5: RR Field Descriptions

Field	Description
Domain name	The domain this record applies to (primary search key)
Time to live (TTL)	How stable the record is. Large value (e.g., 86400=1 day) = stable; small value (e.g., 60=1 min) = volatile
Class	For Internet: always **IN**
Type	What kind of record (SOA, A, AAAA, MX, NS, CNAME, etc.)

Value	Number, domain name, or ASCII string (semantics depend on type)

Type	Meaning	Value
SOA	Start of authority	Parameters for this zone
A	IPv4 address of a host	32-Bit integer
AAAA	IPv6 address of a host	128-Bit integer
MX	Mail exchange	Priority, domain willing to accept email
NS	Name server	Name of a server for this domain
CNAME	Canonical name	Domain name
PTR	Pointer	Alias for an IP address
SPF	Sender policy framework	Text encoding of mail sending policy
SRV	Service	Host that provides it
TXT	Text	Descriptive ASCII text

Figure 7-3. The principal DNS resource record types.

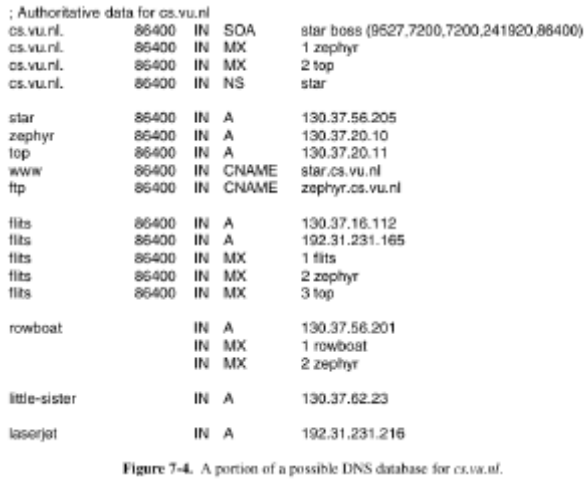
Figure 7.3: *The principal DNS resource record types. SOA (zone authority), A (IPv4 address), AAAA (IPv6 address), MX (mail exchange), NS (name server), CNAME (canonical name/alias), PTR (pointer/reverse lookup), SPF (sender policy), SRV (service), TXT (text).*

Record Type Descriptions

Table 7.6: DNS Resource Record Types in Detail

Type	Meaning	Value
SOA	Start of Authority	Parameters for this zone (admin email, serial number, flags, timeouts)
A	IPv4 address of a host	32-bit integer. Every Internet host must have at least one.
AAAA	IPv6 address of a host	128-bit integer ("Quad A" = 4 A's)
MX	Mail exchange	Priority + domain willing to accept email
NS	Name server	Name of a server for this domain
CNAME	Canonical name	Alias for a domain name
PTR	Pointer	Alias for IP address (reverse lookup)
SPF	Sender Policy Framework	Text encoding of mail sending policy
SRV	Service	Host that provides a given service
TXT	Text	Descriptive ASCII text (often encodes SPF info)

> **CNAME vs PTR**
>
> **CNAME** creates an alias for a **domain name** (e.g., www.cs.vu.nl is an alias for star.cs.vu.nl). **PTR** associates a name with an **IP address** for reverse lookups. Both point to another name, but their interpretations differ.



```
; Authoritative data for cs.vu.nl
cs.vu.nl.       86400   IN  SOA     star boss (9527,7200,7200,241920,86400)
cs.vu.nl.       86400   IN  MX      1 zephyr
cs.vu.nl.       86400   IN  MX      2 top
cs.vu.nl.       86400   IN  NS      star

star            86400   IN  A       130.37.56.205
zephyr          86400   IN  A       130.37.20.10
top             86400   IN  A       130.37.20.11
www             86400   IN  CNAME   star.cs.vu.nl
ftp             86400   IN  CNAME   zephyr.cs.vu.nl

flits           86400   IN  A       130.37.16.112
flits           86400   IN  A       192.31.231.165
flits           86400   IN  MX      1 flits
flits           86400   IN  MX      2 zephyr
flits           86400   IN  MX      3 top

rowboat                 IN  A       130.37.56.201
                        IN  MX      1 rowboat
                        IN  MX      2 zephyr

little-sister           IN  A       130.37.62.23

laserjet                IN  A       192.31.231.216
```

Figure 7-4. A portion of a possible DNS database for cs.vu.nl.

Figure 7.4: *A portion of a possible DNS database for cs.vu.nl. Shows SOA, MX, NS, A, and CNAME records for various hosts including aliases for www and ftp.*

7.1.3 Name Servers

Why Not a Single Name Server?

Table 7.7: Problems with Single Name Server

Problem	Consequence
Overloading	Server would be so overloaded as to be useless
Single point of failure	If it ever went down, the entire Internet would be crippled

Zones

The DNS name space is divided into **nonoverlapping zones**. Each zone contains some part of the tree. Zone boundaries are decided by each zone's administrator.

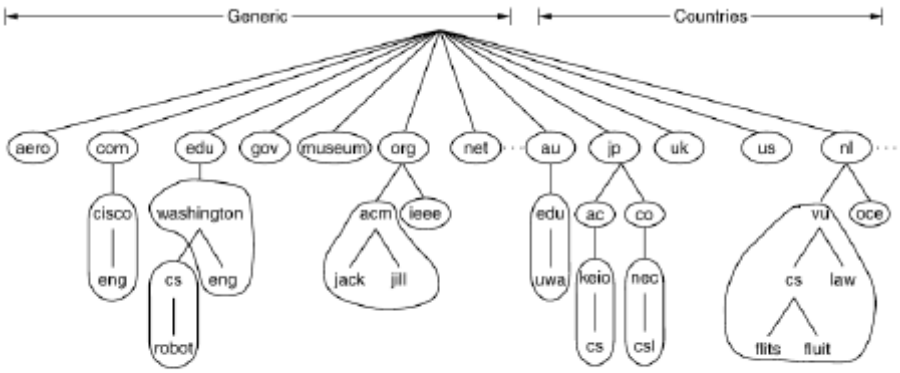


Figure 7-5. Part of the DNS name space divided into zones (which are circled).

Figure 7.5: *Part of the DNS name space divided into zones (circled). washington.edu handles eng.washington.edu but not cs.washington.edu -- CS has its own zone with its own name servers.*

Name Server Types

Table 7.8: Name Server Types

Type	Description
Primary	Gets information from a **file on its disk**
Secondary	Gets information from the **primary name server**

Name Resolution Process

1. Resolver passes query to a **local name server**
2. If the domain falls under the server's jurisdiction: returns **authoritative resource records**
3. If the domain is **remote**: begins a **remote query**

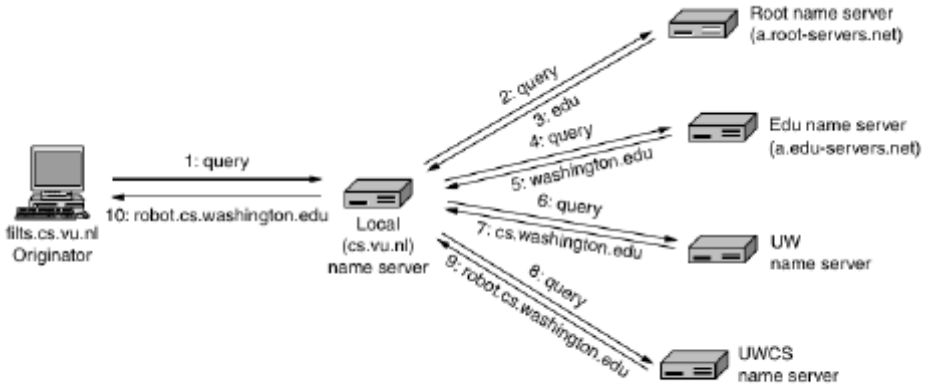


Figure 7-6. Example of a resolver looking up a remote name in 10 steps.

Figure 7.6: *Example of a resolver looking up a remote name in 10 steps. flits.cs.vu.nl queries its local name server, which iteratively queries root, .edu, washington.edu, and finally cs.washington.edu name servers.*

10-Step Remote Query Example

Table 7.9: DNS Resolution Walkthrough (flits.cs.vu.nl -> robot.cs.washington.edu)

Step	Action	Detail
1	Originator -> Local NS	Query contains domain name, type=A, class=IN
2	Local NS -> Root NS	Start at top of hierarchy
3	Root -> Local NS	Returns .edu name server IP
4	Local NS -> .edu NS	Sends full query to .edu server
5	.edu -> Local NS	Returns washington.edu name server
6	Local NS -> UW NS	Sends query to UW name server
7	UW -> Local NS	Returns UW CS name server (cs.washington.edu)
8	Local NS -> UWCS NS	Queries CS name server
9	UWCS -> Local NS	Authoritative answer: robot.cs.washington.edu
10	Local NS -> Originator	Forwards final answer

Two Query Mechanisms

Table 7.10: Recursive vs Iterative Queries

Mechanism	Description	Used By
Recursive	Name server handles full resolution; returns only final answer	Local name server (on behalf of hosts)
Iterative	Returns partial answer (referral); requester continues	Root, TLD, and remote name servers

Caching

All answers, including partial answers, are cached. This greatly reduces query steps and improves performance. The 10-step example is actually the **worst case** (no cached info).

> **Cache Validity**
>
> Cached answers are **not authoritative**. Changes at the source won't propagate automatically. The **TTL field** in each RR tells remote servers how long to cache: stable info = days; volatile info = seconds/minutes.

DNS Transport

Queries and responses use **UDP**. Reliability: if no response arrives, DNS repeats the query, then tries another server. A 16-bit identifier matches answers to queries.

7.2.4 Message Transfer: SMTP

> **SMTP Overview**
>
> **SMTP** (Simple Mail Transfer Protocol) is used by **Message Transfer Agents (MTAs)** to relay messages from originator to recipient. Two uses: **mail submission** (UA -> MTA) and **message transfer** (MTA -> MTA).

Basic SMTP Operation

- Transport: **TCP connection to port 25**
- SMTP is an **ASCII protocol** (easy to develop, test, debug)
- Server listens on port 25, accepts connections, accepts messages, returns error reports for undeliverable mail

SMTP Message Transfer Walkthrough

1. Client establishes TCP connection to port 25; **waits for server to talk first**
2. Server sends greeting with its identity and readiness
3. If unwilling: client disconnects and retries later
4. If willing: mail exchange begins

Figure 7.15: *Sample SMTP dialog. C = client commands, S = server replies with numeric codes (220=ready, 250=ok, 354=send data, 221=closing).*

Command	Purpose
HELO / EHLO	Identify sending host; EHLO initiates ESMTP negotiation
MAIL FROM:	Specify sender's email address
RCPT TO:	Specify recipient (can repeat for multiple recipients)
DATA	Request permission to send message body
. (dot alone)	Signal end of message body
QUIT	Terminate the SMTP session

Table 7.12: SMTP Reply Codes

Code	Meaning	Examples
220	Service ready	220 ee.uwa.edu.au SMTP service ready
221	Service closing	221 ee.uwa.edu.au closing connection
250	Request completed OK	250 sender ok; 250 message accepted
354	Start mail input	354 Send mail; end with "." on a line by itself

Limitations of Basic SMTP

Table 7.13: SMTP Limitations

Limitation	Description	Consequence
No authentication	FROM command can give any sender	Useful for **spam**
ASCII-only	Transfers ASCII, not binary data	Requires **base64 MIME encoding**
No encryption	Sends messages in the clear	No privacy

ESMTP (Extended SMTP)

SMTP was revised with an **extension mechanism** (RFC 5321). Clients send **EHLO** instead of HELO; if accepted, server replies with supported extensions.

Keyword	Description
AUTH	Client authentication
BINARYMIME	Server accepts binary messages
CHUNKING	Server accepts large messages in chunks
SIZE	Check message size before trying to send
STARTTLS	Switch to secure transport (TLS; see Chap. 8)
UTF8SMTP	Internationalized addresses

Figure 7-16. Some SMTP extensions.

Figure 7.16: *Some SMTP extensions. AUTH (client authentication), BINARYMIME (binary messages), CHUNKING (large messages in chunks), SIZE (check size before sending), STARTTLS (secure transport), UTF8SMTP (internationalized addresses).*

Table 7.14: ESMTP Extensions

Keyword	Description
AUTH	Client authentication
BINARYMIME	Server accepts binary messages
CHUNKING	Server accepts large messages in chunks
SIZE	Check message size before trying to send
STARTTLS	Switch to secure transport (TLS)
UTF8SMTP	Internationalized addresses

Mail Submission vs Message Transfer

Table 7.15: Mail Submission vs Message Transfer

Aspect	Mail Submission	Message Transfer
Purpose	User agent -> MTA	MTA -> MTA
Port	**587** (preferred)	25
Authentication	Required (AUTH)	Not always required
Message checking	Server checks/corrects format	Less checking
Standard	RFC 4409	RFC 5321

Multipart MIME Messages

```
From: alice@cs.washington.edu
To: bob@ee.uwa.edu.au
MIME-Version: 1.0
Message-Id: <0704760941.AA00747@cs.washington.edu>
Content-Type: multipart/alternative; boundary=qwertyuiopasdfghjklzxcvbnm
Subject: Earth orbits sun integral number of times

This is the preamble. The user agent ignores it. Have a nice day.

--qwertyuiopasdfghjklzxcvbnm
Content-Type: text/html

<p>Happy birthday to you<br>
Happy birthday to you<br>
Happy birthday dear <b> Bob </b><br>
Happy birthday to you</p>

--qwertyuiopasdfghjklzxcvbnm
Content-Type: message/external-body;
        access-type="anon-ftp";
        site="bicycle.cs.washington.edu";
        directory="pub";
        name="birthday.snd"

content-type: audio/basic
content-transfer-encoding: base64
--qwertyuiopasdfghjklzxcvbnm--
```

Figure 7-14. A multipart message containing HTML and audio alternatives.

Figure 7.14: *A multipart message containing HTML and audio alternatives. Uses boundary string to separate parts. Content-Type: multipart/alternative lets the recipient choose the best format.*

7.3 The World Wide Web

Overview and History

> **Definition**
>
> The Web is an **architectural framework for accessing linked content** spread over millions of machines. Invented at **CERN in 1989** by **Tim Berners-Lee**.

Table 7.16: Web Timeline

Year	Milestone
1989	Proposal made at CERN
~1990	First text-based prototype operational
1991	Public demonstration at Hypertext '91
1993	**Mosaic** -- first graphical browser (Marc Andreessen)
1994	Netscape formed; W3C established
1994-1997	Browser War: Netscape Navigator vs Internet Explorer

7.3.1 Architectural Overview

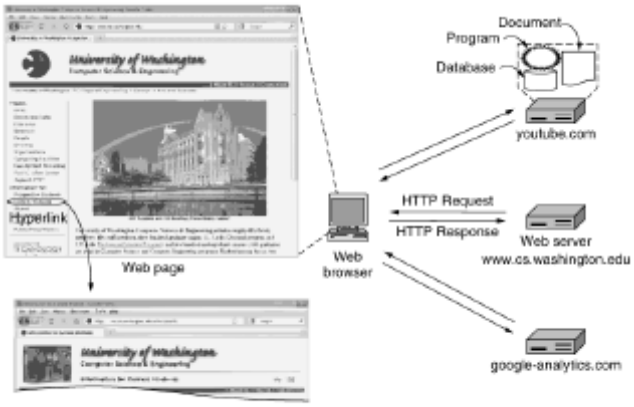


Figure 7-18. Architecture of the Web.

Figure 7.18: *Architecture of the Web. The browser contacts multiple servers (main page server, YouTube for video, Google Analytics for tracking) and integrates content for display. Users cannot tell where content originates.*

Steps in Fetching a Web Page

Table 7.17: Page Fetch Steps

Step	Action
1	Browser determines the URL from the clicked hyperlink
2	Browser asks DNS for IP address of www.cs.washington.edu
3	DNS replies with 128.208.3.88
4	Browser makes TCP connection to 128.208.3.88 on **port 80**
5	Sends HTTP request asking for /index.html
6	Server sends page as HTTP response
7	Browser fetches other embedded URLs
8	Browser displays the complete page
9	TCP connections released if idle

URLs (Uniform Resource Locators)

Table 7.18: URL Components

Part	Description	Example
Protocol (scheme)	Protocol to use	http, https, ftp
DNS name of host	Machine hosting the page	www.cs.washington.edu
Path	Specific page identifier	index.html

Name	Used for	Example
http	Hypertext (HTML)	http://www.ee.uwa.edu/~rob/
https	Hypertext with security	https://www.bank.com/accounts/
ftp	FTP	ftp://ftp.cs.vu.nl/pub/minix/README
file	Local file	file:///usr/suzanne/prog.c
mailto	Sending email	mailto:JohnUser@acm.org
rtsp	Streaming media	rtsp://youtube.com/montypython.mpg
sip	Multimedia calls	sip:eve@adversary.com
about	Browser information	about:plugins

Figure 7-19. Some common URL schemes.

Figure 7.19: *Some common URL schemes. http (hypertext), https (secure), ftp (file transfer), file (local file), mailto (email), rtsp (streaming), sip (multimedia calls), about (browser info).*

Static vs Dynamic Pages

Table 7.19: Page Types

Type	Definition	Example
Static	Same every time displayed	Simple HTML file
Dynamic	Generated on demand by a program	Personalized store front page

URI vs URL vs URN

Table 7.20: URI/URL/URN Distinction

Term	Full Name	Function
URL	Uniform Resource Locator	Tells **how to locate** a resource (protocol + host + path)

URN	Uniform Resource Name	Tells the **name only** -- no location info
URI	Uniform Resource Identifier	General term encompassing both URLs and URNs

7.3.4 HTTP -- The HyperText Transfer Protocol

> **HTTP Overview**
>
> HTTP (RFC 2616) is a **simple request-response protocol** that runs over **TCP**. Request/response headers in **ASCII**; contents in **MIME-like format**.

Connections: HTTP 1.0 vs HTTP 1.1

Table 7.21: HTTP Version Comparison

Aspect	HTTP 1.0	HTTP 1.1
Connections	One request per connection, then close	**Persistent connections** (connection reuse)
Pipelining	No	Yes
Performance	Poor for pages with many objects	Better -- amortizes TCP overhead
Congestion control	Multiple connections compete	Single connection avoids competition

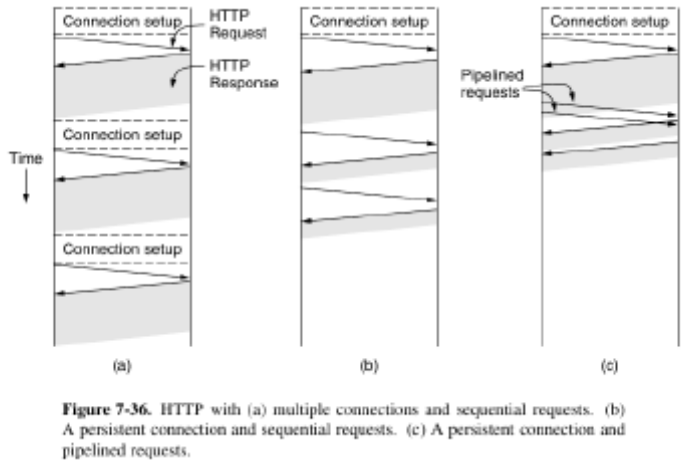


Figure 7-36. HTTP with (a) multiple connections and sequential requests. (b) A persistent connection and sequential requests. (c) A persistent connection and pipelined requests.

Figure 7.36: *HTTP connection strategies. (a) Multiple connections, sequential requests -- slowest. (b) Persistent connection, sequential requests -- faster. (c) Persistent connection with pipelined requests -- fastest.*

> **Why Persistent Connections Are Faster**
>
> 1. **No connection setup overhead:** Each TCP connection needs at least one RTT to establish
> 2. **Better congestion control:** Multiple short TCP connections each go through slow-start; one longer connection learns the network path once

HTTP Methods

Method	Description
GET	Read a Web page
HEAD	Read a Web page's header
POST	Append to a Web page
PUT	Store a Web page
DELETE	Remove the Web page
TRACE	Echo the incoming request
CONNECT	Connect through a proxy
OPTIONS	Query options for a page

Figure 7-37. The built-in HTTP request methods.

Figure 7.37: *The built-in HTTP request methods. GET (read), HEAD (read header), POST (append/submit), PUT (store), DELETE (remove), TRACE (echo/debug), CONNECT (proxy), OPTIONS (query capabilities).*

Table 7.22: HTTP Methods Detail

Method	Description	Safe?	Idempotent?
GET	Read a Web page -- **vast majority** of requests	Yes	Yes
HEAD	Read a page's header only (for indexing/testing)	Yes	Yes
POST	Append/submit data (forms, SOAP)	No	No
PUT	Store a page on remote server	No	Yes
DELETE	Remove a page (needs auth)	No	Yes
TRACE	Echo the incoming request (debugging)	Yes	Yes
CONNECT	Connect through a proxy	No	No
OPTIONS	Query options for a page	Yes	Yes

> **Safe vs Idempotent**
>
> **Safe methods** do not modify server state (GET, HEAD, TRACE, OPTIONS). **Idempotent methods** produce the same result if executed multiple times (GET, HEAD, PUT, DELETE, TRACE, OPTIONS). POST is neither safe nor idempotent.

HTTP Status Codes

Code	Meaning	Examples
1xx	Information	100 = server agrees to handle client's request
2xx	Success	200 = request succeeded; 204 = no content present
3xx	Redirection	301 = page moved; 304 = cached page still valid
4xx	Client error	403 = forbidden page; 404 = page not found
5xx	Server error	500 = internal server error; 503 = try again later

Figure 7-38. The status code response groups.

Figure 7.38: *The status code response groups. 1xx=Information, 2xx=Success, 3xx=Redirection, 4xx=Client error, 5xx=Server error.*

Table 7.23: HTTP Status Code Groups

Code	Meaning	Examples
1xx	Information	100 = server agrees to handle request
2xx	Success	200 = request succeeded; 204 = no content
3xx	Redirection	301 = page moved; 304 = cached page still valid
4xx	Client error	403 = forbidden; **404 = page not found**
5xx	Server error	500 = internal server error; 503 = try again later

Quick Reference Summary Tables

Transport vs Network Layer

Summary 1: Transport Layer vs Network Layer

Aspect	Transport Layer	Network Layer
Primary goal	Efficient, reliable service to application processes	Packet delivery across networks
Entity location	User machines (end systems)	Routers (carrier-operated)
User control	High	Low / None
Service types	Connection-oriented, Connectionless	Connection-oriented, Connectionless
Key improvement	Can be more reliable than underlying network	Best-effort delivery
Boundary role	Provider (layers 1-4) vs User (layers 5+)	Part of service provider

UDP vs TCP

Summary 2: UDP vs TCP Comparison

Feature	UDP	TCP
Type	Connectionless	Connection-oriented
Header size	8 bytes	20 bytes (fixed) + up to 40 bytes options
Flow control	No	Yes (sliding window)
Congestion control	No	Yes
Reliability	No retransmission	Retransmission with timers
Checksum	Optional	Mandatory
Sequencing	No	32-bit per-byte sequence numbers

Demultiplexing	Source + Destination ports	Source + Destination ports
Typical use	DNS, streaming, games	HTTP, FTP, email

DNS Record Types Quick Reference

Summary 3: DNS Resource Records Summary

Type	Purpose	Value
SOA	Zone authority info	Zone parameters
A	IPv4 address	32-bit integer
AAAA	IPv6 address	128-bit integer
MX	Mail server	Priority + domain
NS	Name server	Server name
CNAME	Alias	Canonical domain name
PTR	Reverse lookup	Alias for IP address
SRV	Service location	Host name
TXT	Text/SPF	ASCII string

HTTP Connection Strategies

Summary 4: HTTP Connection Strategies Compared

Strategy	Connections	Requests	Pros	Cons
Multiple sequential	Multiple (one per object)	Sequential	Simple	High overhead; slow-start penalty per connection
Persistent sequential	One	Sequential	Reduced overhead	Server idle between requests
Persistent pipelined	One	Pipelined	Fastest; minimal idle time	More complex

Parallel	Multiple simultaneous	One per connection	Hides some latency	Connections compete; discouraged

Exam Practice: Questions & Answers

How to Use This Section

These questions cover both **theoretical concepts** and **numerical problems** commonly asked in exams. Try answering each question on your own before checking the answer. Numerical problems include step-by-step solutions.

Section A: Theoretical Questions

1 What is the primary objective of the transport layer?

Answer: The transport layer aims to provide efficient, reliable, and cost-effective data transmission service to application-layer processes. It achieves this by using the services of the network layer while isolating upper layers from network imperfections.

2 Why do we need a separate transport layer if the network layer already provides similar services?

Answer: (1) The transport layer runs on users' machines where users have full control; the network layer runs on carrier-operated routers where users have no control. (2) The transport layer can compensate for poor network service through retransmissions and connection recovery. (3) It isolates applications from network technology differences -- applications work across Ethernet, WiMAX, etc. without modification.

3 What is the transport entity, and where can it be located?

Answer: The transport entity is the software/hardware that performs data transmission in the transport layer. Locations: (1) OS kernel -- most common, (2) Library package bound into applications -- most common, (3) Separate user process, (4) Network interface card -- less common.

4 **Differentiate between connection-oriented and connectionless transport services.**

Answer: Connection-oriented has three phases: establishment, data transfer, and release. It uses addressing and flow control similar to connection-oriented network service. Connectionless is very similar to connectionless network service -- each packet is independent. It is difficult to provide connectionless transport over connection-oriented networks because setting up a connection for a single packet is inefficient.

5 **Why was DNS invented? What were the problems with hosts.txt?**

Answer: DNS was invented in 1983 because hosts.txt failed to scale: (1) File size became too large for millions of hosts, (2) Name conflicts occurred without central management, (3) Central management was infeasible for a global network due to load and latency. DNS uses a hierarchical, distributed approach that scales to the entire Internet.

6 **Explain the DNS resolver mechanism with steps.**

Answer: (1) Application calls the resolver library procedure, passing the domain name. (2) Resolver sends a UDP query containing the name to a local DNS server. (3) Local DNS server looks up the name and returns the IP address. (4) Resolver returns the IP address to the application. The application can then establish a TCP connection or send UDP packets.

7 **What is the difference between recursive and iterative DNS queries?**

Answer: In a **recursive query**, the name server handles the full resolution on behalf of the client and returns only the final answer. Used by local name servers. In an **iterative query**, the name server returns a partial answer (referral to another server), and the requester must continue the resolution. Used by root and TLD servers to avoid overloading them.

8 Explain why caching is important in DNS and how TTL controls it.

Answer: Caching greatly reduces query steps and improves performance. All answers, including partial referrals, are cached. However, cached answers may become out of date. The **TTL (Time To Live)** field in each resource record tells remote servers how long to cache it: stable information (e.g., a machine's IP that hasn't changed in years) can have large TTL (e.g., 86400 = 1 day); volatile information should have small TTL (seconds or minutes).

9 What is the purpose of MX records? Why are they needed?

Answer: MX (Mail Exchange) records specify which host is prepared to accept email for a domain. They are needed because not every machine in a domain is configured to receive email. MX records include a priority value so multiple mail servers can be listed in order of preference.

10 Differentiate between CNAME and PTR records.

Answer:CNAME (Canonical Name) creates an alias for a domain name (e.g., www.cs.vu.nl -> star.cs.vu.nl). It is primarily used for convenience when services move. **PTR (Pointer)** is used for reverse lookups -- mapping an IP address back to a domain name. Both point to another name but serve different purposes.

11 Describe the SMTP mail transfer process step by step.

Answer: (1) Sending machine establishes TCP connection to port 25 of receiving machine. (2) Server sends greeting first (220 code). (3) Client sends HELO/EHLO to identify itself. (4) Client sends MAIL FROM: with sender address. (5) Client sends RCPT TO: with recipient address. (6) Client sends DATA command. (7) Server responds with 354 (ready for data). (8) Client sends message body, ending with a dot on its own line. (9) Server acknowledges with 250. (10) Client sends QUIT; server closes connection with 221.

12 What are the limitations of basic SMTP, and how does ESMTP address them?

Answer: Basic SMTP limitations: (1) No authentication -- FROM can be faked (spam), (2) ASCII-only -- requires base64 for binary, (3) No encryption -- no privacy. ESMTP addresses these through extensions: AUTH (authentication), BINARYMIME (binary data), STARTTLS (encryption), SIZE (message size checking), CHUNKING (large messages), UTF8SMTP (internationalized addresses).

13 Compare mail submission and message transfer in terms of port, authentication, and purpose.

Answer:Mail submission (UA to MTA): uses port 587, requires AUTH authentication, server checks/corrects message format, defined in RFC 4409. **Message transfer** (MTA to MTA): uses port 25, authentication not always required, less format checking, defined in RFC 5321.

14 Explain the difference between static and dynamic web pages.

Answer: A **static page** is the same every time it is displayed (e.g., a simple HTML file). A **dynamic page** is generated on demand by a program running on the server, and its content may vary per user or per request (e.g., a personalized storefront showing different products to different customers).

15 Explain URL, URI, and URN with differences.

Answer:URL (Uniform Resource Locator) specifies both the name and location of a resource (protocol + host + path). **URN (Uniform Resource Name)** specifies only the name without location information, allowing content to be found from any available copy. **URI (Uniform Resource Identifier)** is the general term encompassing both URLs and URNs.

16 What is HTTP pipelining, and why is it faster?

Answer: HTTP pipelining allows a client to send multiple HTTP requests on a persistent connection without waiting for each response. It is faster because: (1) No connection setup overhead per object, (2) TCP congestion control (slow-start) only happens once on a single connection rather than for multiple short connections, (3) Server idle time is minimized.

17 What are safe and idempotent HTTP methods? Give examples.

Answer:Safe methods do not modify server state: GET, HEAD, TRACE, OPTIONS. **Idempotent methods** produce the same result when executed multiple times: GET, HEAD, PUT, DELETE, TRACE, OPTIONS. POST is neither safe nor idempotent. PUT and DELETE are idempotent but not safe.

18 Explain the 5-tuple that uniquely identifies a TCP connection.

Answer: The 5-tuple consists of: (1) Protocol (TCP), (2) Source IP address, (3) Source port number, (4) Destination IP address, (5) Destination port number. Together these five values uniquely identify a single TCP connection between two hosts.

19 What is the three-way handshake in TCP, and which flags are used?

Answer: TCP connection establishment uses a three-way handshake: (1) Client sends SYN=1, ACK=0 (connection request), (2) Server replies with SYN=1, ACK=1 (connection accepted + acknowledgement), (3) Client sends ACK=1 to acknowledge the server's SYN. The SYN flag means "synchronize sequence numbers," and ACK means "acknowledgement number is valid."

20 What is the purpose of the TCP Window Scale option? When is it needed?

Answer: The Window Scale option allows TCP to overcome the 64 KB limit of the 16-bit Window Size field. It lets both sides negotiate a scale factor (up to 14 bits left shift), enabling window sizes up to 2^30 bytes (~1 GB). It is needed on high-bandwidth, high-delay networks (long fat pipes) where the bandwidth-delay product exceeds 64 KB.

Section B: Numerical Problems

N1 A TCP connection uses the Window Scale option with a scale factor of 5. If the Window Size field in the TCP header shows 4000, what is the actual advertised window size in bytes?

Answer: Actual window = Window Size field x 2^scale_factor = 4000 x 2^5 = 4000 x 32 = **128,000 bytes**.

N2 A host wants to send data at 1 Gbps over a connection with 100 ms RTT. What minimum window size is needed to keep the pipe full? Does the basic TCP window (16-bit, max 64 KB) suffice? If not, what scale factor is needed?

Answer: Bandwidth-delay product = 1 Gbps x 100 ms = 10^9 x 0.1 = **12.5 MB**.
Max basic window = 2^16 - 1 = 65,535 bytes = ~64 KB. This is insufficient.
Scale factor needed: 12.5 MB / 64 KB = ~200, so we need 2^scale >= 200. Since 2^7 = 128 and 2^8 = 256, we need a **scale factor of at least 8**.

N3 A UDP datagram has 2000 bytes of data. What is the total size of the UDP segment? What is the value in the UDP Length field?

Answer: UDP header = 8 bytes. Data = 2000 bytes. Total segment size = 2008 bytes. The UDP Length field contains **2008** (header + data).

N4 What is the maximum amount of data that can be carried in a single TCP segment? Show the calculation.

Answer: Maximum IP packet = 65,535 bytes. IP header (minimum) = 20 bytes. TCP header (minimum) = 20 bytes. Max TCP data = 65,535 - 20 - 20 = **65,495 bytes**.

N5 A DNS A record has a TTL of 86400 seconds. How long will a caching name server keep this record? Convert this to hours and explain what it indicates about the stability of the record.

Answer: TTL = 86400 seconds = 86400 / 3600 = **24 hours (1 day)**. This large TTL indicates the information is **highly stable** -- the IP address is unlikely to change, so it is safe to cache for a long duration.

N6 What is the maximum length of a domain name component? What is the maximum length of a full domain path name?

Answer: Each component (label) can be up to **63 characters**. The full domain name path must not exceed **255 characters** (including dots).

N7 A TCP sender has sent bytes 0 through 9999. It receives an acknowledgement with Ack number = 8000 and Window size = 5000. How many more bytes can it send? Explain.

Answer: Ack number 8000 means bytes 0-7999 are acknowledged. The sender can send from byte 8000 onward. Window size 5000 means up to 5000 bytes starting at 8000. So the sender can send bytes **8000 through 12999** = **5000 bytes** total.

N8 An HTTP 1.0 client needs to fetch a page with the main HTML file and 5 embedded images, all from the same server. How many TCP connections are needed? If each connection setup takes 1 RTT, what is the total connection setup time in RTTs?

Answer: HTTP 1.0 uses one connection per object. Total objects = 1 HTML + 5 images = 6 objects. So **6 TCP connections** are needed. Total connection setup time = 6 x 1 RTT = **6 RTTs**. (In HTTP 1.1 with persistent connections, this would be just 1 RTT.)

N9 A UDP segment has a checksum field value of 0xFFFF (all 1s). What does this indicate? What if the value were 0x0000?

Answer: 0xFFFF (all 1s) means the checksum **was computed** and the result happened to be 0 (in 1's complement, 0 is stored as all 1s). 0x0000 means the checksum **was not computed** -- the sender skipped checksum calculation. This is unambiguous due to the properties of 1's complement arithmetic.

N10 At 56 kbps (original ARPANET speed), how long does it take to cycle through all 32-bit TCP sequence numbers? At 1 Gbps, how long does it take?

Answer: Total sequence numbers = 2^32 = 4,294,967,296 bytes.
At 56 kbps = 7,000 bytes/sec. Time = 4,294,967,296 / 7,000 = 613,567 seconds = **~7.1 days (~1 week)**.
At 1 Gbps = 125,000,000 bytes/sec. Time = 4,294,967,296 / 125,000,000 = **~34.4 seconds**.
This shows why sequence number wraparound (PAWS, timestamps) became critical at modern speeds.

N11 A DNS name server receives a query for www.example.com. It has cached the .com name server address but nothing else. How many query steps are needed to resolve the name? List them.

Answer: Since the .com server is cached, we skip root. Steps: (1) Local NS queries .com NS, (2) .com NS returns example.com NS, (3) Local NS queries example.com NS, (4) example.com NS returns the A record for www.example.com, (5) Local NS returns answer to client. Total = **5 steps** (vs. 10 steps with no cached info).

N12 If a TCP connection uses the MSS option with value 1460, and the IP header is 20 bytes, what is the total size of each segment? How many segments are needed to send 100 KB of data?

Answer: MSS = 1460 bytes (payload). TCP header = 20 bytes. IP header = 20 bytes. Total per segment = 1460 + 20 + 20 = **1500 bytes**.
Data to send = 100 KB = 102,400 bytes. Number of segments = ceil(102,400 / 1460) = ceil(70.14) = **71 segments**. The last segment carries 102,400 - (70 x 1460) = 102,400 - 102,200 = 200 bytes.

N13 Calculate the UDP checksum for a packet where the 16-bit words (including header, zero-padded data, and pseudoheader) sum to 0x12345 in 1's complement arithmetic. What value goes in the checksum field?

Answer: In 1's complement, a 5-digit sum 0x12345 wraps around: add the carry back: 0x2345 + 0x1 = 0x2346. Checksum = 1's complement of 0x2346 = **0xDCB9**. The receiver adds all 16-bit words including checksum; the result should be 0xFFFF (which represents 0 in 1's complement).

N14 An SMTP server responds with code "354". What does this mean? A client then sends a dot on a line by itself. What response code should it expect?

Answer: Code **354** means "Start mail input; end with CRLF.CRLF" -- the server is ready to receive the message body. After the client sends the terminating dot on its own line, the server should respond with **250** (message accepted/requested action completed OK).

N15 A web page has 1 HTML file, 8 images, 2 CSS files, and 1 JavaScript file -- all from the same server. Compare the total connection setups needed for HTTP 1.0 vs HTTP 1.1 (persistent).

Answer: Total objects = 1 + 8 + 2 + 1 = 12 objects.
HTTP 1.0: 12 TCP connections (one per object) = 12 connection setups.
HTTP 1.1: 1 TCP connection (persistent, reused) = 1 connection setup.
Savings = 11 connection setups. Each setup requires at least 1 RTT, so HTTP 1.1 saves at least **11 RTTs** of setup time.

Final Revision Checklist

- Transport entity locations and the provider-user distinction (layers 1-4 vs 5+)
- UDP header format (8 bytes: 4 fields of 16 bits each) and checksum calculation
- IPv4 pseudoheader composition (src IP, dst IP, pad, protocol, UDP length)
- TCP header all fields, especially the 8 flags and their meanings
- 5-tuple connection identification
- TCP options: MSS, Window Scale, Timestamp, SACK
- DNS hierarchical name space, zone delegation, ICANN role
- DNS resolution: recursive vs iterative, 10-step example
- All RR types: SOA, A, AAAA, MX, NS, CNAME, PTR, SRV, TXT
- SMTP command sequence with numeric codes (220, 250, 354, 221)
- ESMTP extensions and their purposes
- Mail submission (port 587, AUTH) vs message transfer (port 25)
- URL components, URI/URN distinction
- HTTP methods (safe, idempotent), status code groups (1xx-5xx)
- HTTP connection strategies: multiple, persistent, pipelined, parallel